



الجامعة الافتراضية السورية

البرنامج: TIC

الفضل: S25

19.6.

الجامعة الافتراضية السورية
SYRIAN VIRTUAL UNIVERSITY



IPG 203

OBJECT ORIENTED PROGRAMMING

Dr. Jihad Ali Issa

الاسم الكامل	Name	Username	<u>UserID</u>	Class
حمزة لؤي عمران	Hamzah Omran	hamzah	357401	C5
بتول عدنان شوره	Batool Shouraba	batool	257341	C5

الفهرس

٣	مقدمة
٣	فكرة المشروع
٤	تصميم المشروع
٤	الواجهة IProductActions :
٤	الفئة المجردة Product :
٥	الفئات المشتقة من Product :
٥	Electronics
٥	Clothing
٥	Food
٦	الفئة Store :
٦	الفئة الساكنة Validator :
٧	تطبيق مبادئ البرمجة غرضية التوجه
٧	التجريد (Abstraction) :
٧	الوراثة (Inheritance) :
٧	تعدد الأشكال (Polymorphism) :
٧	التغليف (Encapsulation) :
٧	التفويض والأحداث (Delegates & Events) :
٨	الفئات والعناصر الساكنة (Static Classes & Members) :
٨	تشغيل المشروع
٩	مخطط UML
١٠	خاتمة
١٠	رابط المشروع على GitHub

مقدمة

يهدف هذا المشروع إلى تطوير نظام بسيط لإدارة متجر باستخدام لغة C# مع تطبيق المبادئ الأساسية للبرمجة غرضية التوجه (Object Oriented Programming) تم تصميم النظام بطريقة تسمح بإدارة أنواع مختلفة من المنتجات ضمن بنية مرنة وقابلة للتوسع، مع التركيز على تطبيق مفاهيم التجريد، الوراثة، التغليف، تعدد الأشكال، والأحداث بصورة عملية وواضحة.

يسمح النظام بإضافة المنتجات إلى المتجر، عرض معلوماتها، حساب السعر النهائي لكل منتج وفق نوعه، وتنفيذ عمليات البيع مع إرسال إشعارات عند حدوث حالات محددة مثل إضافة منتج جديد أو نفاذ الكمية من المخزون.

فكرة المشروع

يعتمد المشروع على إنشاء متجر يحتوي على عدة أنواع من المنتجات، حيث تم تقسيم المنتجات إلى أصناف مختلفة تشمل:

- المنتجات الإلكترونية (Electronics)

- الملابس (Clothing)

- المنتجات الغذائية (Food)

يملك كل نوع خصائصه الخاصة بالإضافة إلى الخصائص المشتركة الموجودة في جميع المنتجات، مثل المعرّف والاسم والسعر والكمية.

يقوم النظام بتنفيذ مجموعة من العمليات الأساسية، مثل:

- إضافة المنتجات إلى المتجر

- عرض معلومات المنتجات

- حساب السعر النهائي لكل منتج

- بيع المنتجات وإنقاص الكمية

- إرسال إشعارات عند إضافة منتج أو نفاذ الكمية

تصميم المشروع

الواجهة IProductActions :

تم إنشاء واجهة باسم IProductActions تحتوي على العمليات المشتركة التي يجب أن تطبقها جميع أنواع المنتجات، وهي:

- `DisplayInfo()`: دالة لعرض معلومات المنتج
- `CalculateFinalPrice()`: دالة لعرض السعر النهائي للمنتج

ساهمت هذه الواجهة في تحقيق مبدأ التجريد من خلال فرض مجموعة من السلوكيات المشتركة على جميع المنتجات.

الفئة المجردة Product :

تم إنشاء فئة مجردة باسم Product لتحتوي الخصائص المشتركة بين جميع المنتجات، وتشمل:

- `Id`
- `Name`
- `Price`
- `Quantity`

كما تحتوي الفئة على:

- خاصية ساكنة `TotalProducts`
- دالة مجردة `CalculateFinalPrice()`
- دالة `DisplayInfo()`

تم استخدام الفئة المجردة لتجنب تكرار الخصائص والوظائف المشتركة ضمن الفئات الفرعية.

الفئات المشتقة من Product:

Electronics

تمثل المنتجات الإلكترونية، وتحتوي على خاصية إضافية:

- WarrantyMonths: خاصية لإضافة ضريبة على المنتج

كما تقوم بإعادة تعريف الدالة CalculateFinalPrice() لإضافة ضريبة على السعر الأساسي.

Clothing

تمثل منتجات الملابس، وتحتوي على الخصائص التالية:

- Size
- Material

كما تعيد تعريف دالة حساب السعر النهائي لتطبيق نسبة حسم معينة.

Food

تمثل المنتجات الغذائية، وتحتوي على الخاصية:

- ExpiryDate

كما تقوم بإعادة تعريف دالة حساب السعر النهائي بطريقة مختلفة عن بقية الأنواع.

الفئة Store :

تمثل هذه الفئة المتجر الرئيسي المسؤول عن إدارة المنتجات.

تحتوي على قائمة من النوع:

List<Product>

مما يسمح بتخزين أنواع مختلفة من المنتجات داخل نفس القائمة وتطبيق مبدأ تعدد الأشكال.

كما تحتوي على مجموعة من الدوال:

- AddProduct()
- DisplayProducts()
- SellProduct()

الفئة الساكنة Validator :

تم إنشاء فئة ساكنة باسم Validator للتحقق من صحة البيانات المدخلة، وتتضمن الدوال التالية:

- IsValidName()
- IsValidPrice()
- IsValidQuantity()

تم استخدام هذه الفئة للتحقق من البيانات قبل إنشاء المنتجات وإضافتها إلى المتجر.

تطبيق مبادئ البرمجة غرضية التوجه

التجريد (Abstraction) :

تم تطبيق التجريد باستخدام الواجهة ProductActions والفئة المجردة Product، حيث تم تعريف العمليات المشتركة بين جميع المنتجات دون الحاجة لمعرفة طريقة التنفيذ التفصيلية داخل كل نوع.

الوراثة (Inheritance) :

تم تطبيق الوراثة من خلال جعل الفئات:

- Electronics
- Clothing
- Food

ترث من الفئة المجردة Product للاستفادة من الخصائص والوظائف المشتركة.

تعدد الأشكال (Polymorphism) :

تم تطبيق تعدد الأشكال باستخدام القائمة: List<Product> داخل الفئة Store، حيث يمكن التعامل مع جميع أنواع المنتجات بطريقة موحدة رغم اختلاف تنفيذ الدوال داخل كل فئة فرعية.

التغليف (Encapsulation) :

تم جعل جميع الحقول خاصة private مع استخدام الخصائص Properties للتحكم بالوصول إلى البيانات ومنع التعديل المباشر عليها.

كما تم جعل المعرف Id للقراءة فقط لمنع تعديله بعد إنشاء الكائن.

التفويض والأحداث (Delegates & Events) :

تم إنشاء Delegate باسم NotificationHandler واستخدامه ضمن أحداث داخل الفئة Store مثل:

- ProductAdded
- OutOfStock

حيث يتم إرسال إشعارات عند إضافة منتج جديد أو عند نفاد الكمية من المخزون.

الفئات والعناصر الساكنة (Static Classes & Members) :

تم إنشاء الفئة الساكنة Validator لاستخدامها في التحقق من صحة البيانات.

كما تم استخدام الخاصية الساكنة TotalProducts داخل الفئة Product لحساب عدد المنتجات المنشأة داخل النظام.

تشغيل المشروع

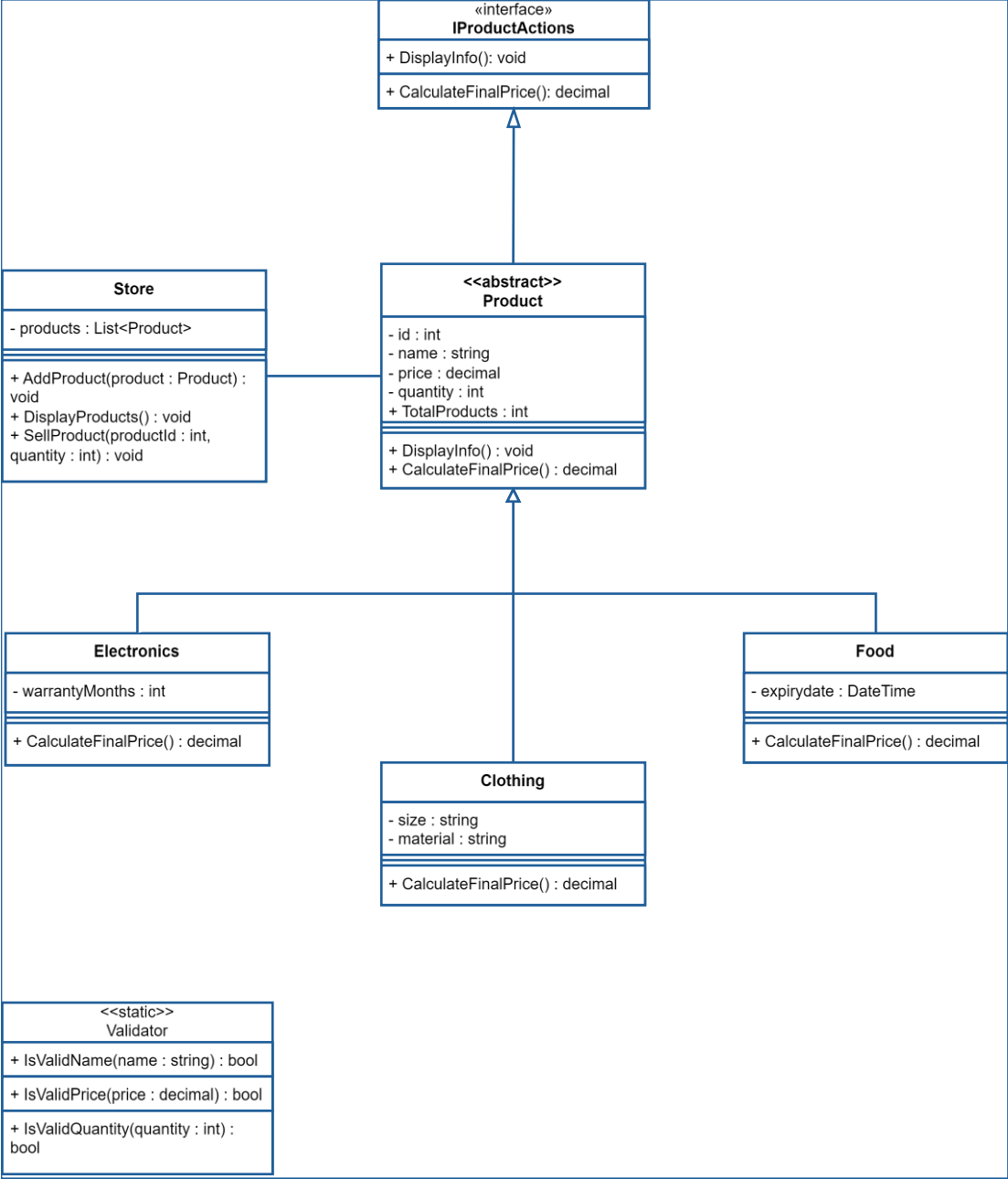
عند تشغيل البرنامج يتم إنشاء متجر جديد وإضافة مجموعة من المنتجات المختلفة إليه.

يقوم النظام بعد ذلك بعرض معلومات جميع المنتجات مع حساب السعر النهائي لكل منتج وفق نوعه، ثم يتم تنفيذ عمليات بيع لبعض المنتجات.

عند نفاد كمية أحد المنتجات يتم إطلاق حدث OutOfStock وإظهار رسالة تنبيه للمستخدم.

كما يتم عرض العدد الإجمالي للمنتجات التي تم إنشاؤها باستخدام الخاصية الساكنة TotalProducts.

مخطط UML



يوضح مخطط UML العلاقات الأساسية بين الفئات المستخدمة في المشروع، بما في ذلك علاقات الوراثة والتجريد والارتباط بين الكائنات المختلفة.

خاتمة

تم في هذا المشروع تطوير نظام مبسط لإدارة متجر باستخدام مبادئ البرمجة غرضية التوجه، مع التركيز على التطبيق العملي للمفاهيم الأساسية.

ساهم استخدام هذه المبادئ في بناء مشروع منظم وقابل للتوسع وسهل الصيانة، كما أظهر أهمية البرمجة غرضية التوجه في تنظيم التطبيقات البرمجية وتقليل التكرار وتحسين إعادة استخدام الكود.

رابط المشروع على GitHub

[HazzDevv/StoreManagementSystem: Homework for subject IPG203 \(Object Oriented Programming\) in SVU \(Syrian Virtual University\) TIC \(Technology Institute for Computers\)](#)